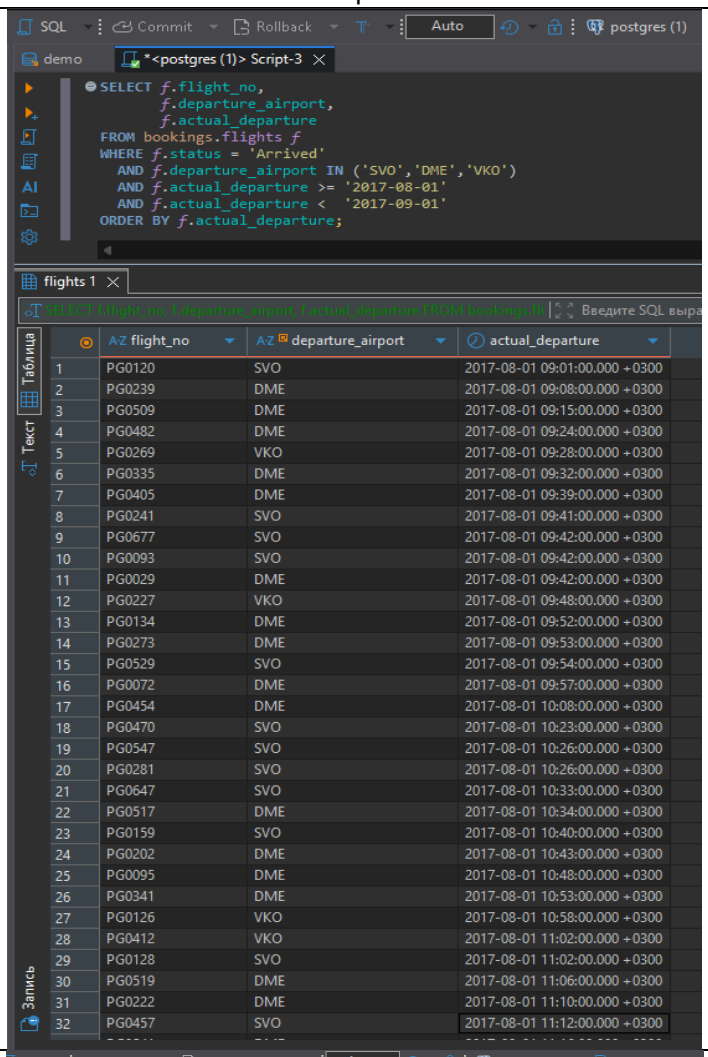
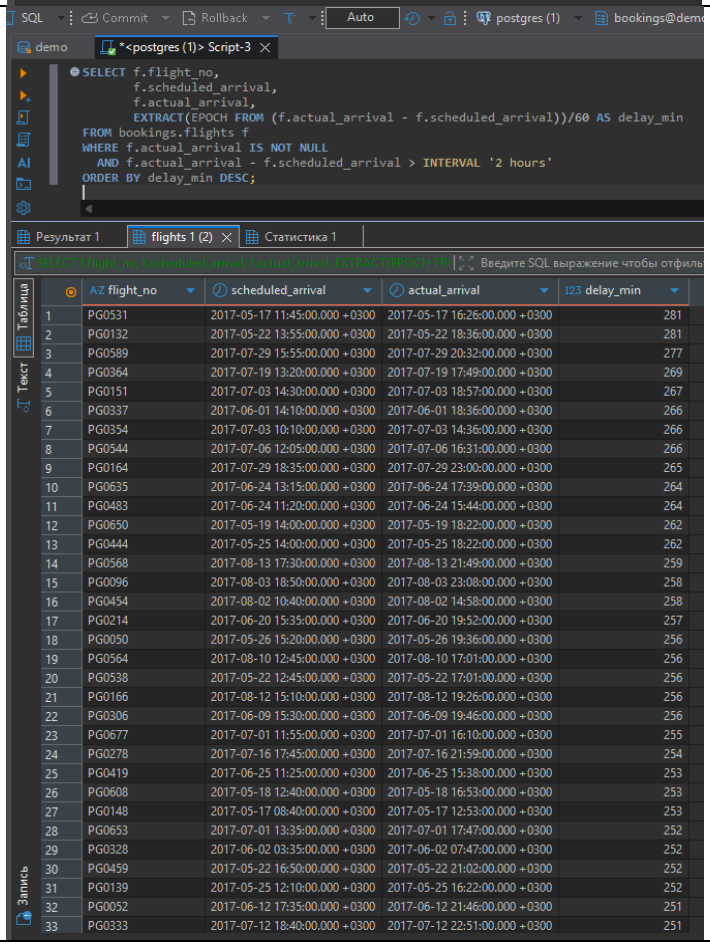
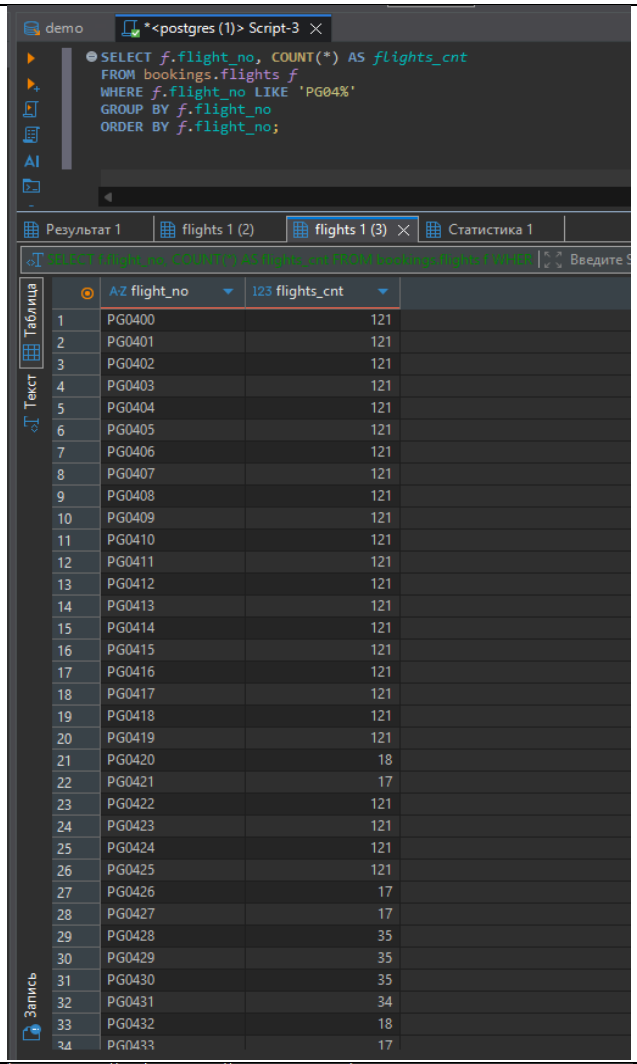
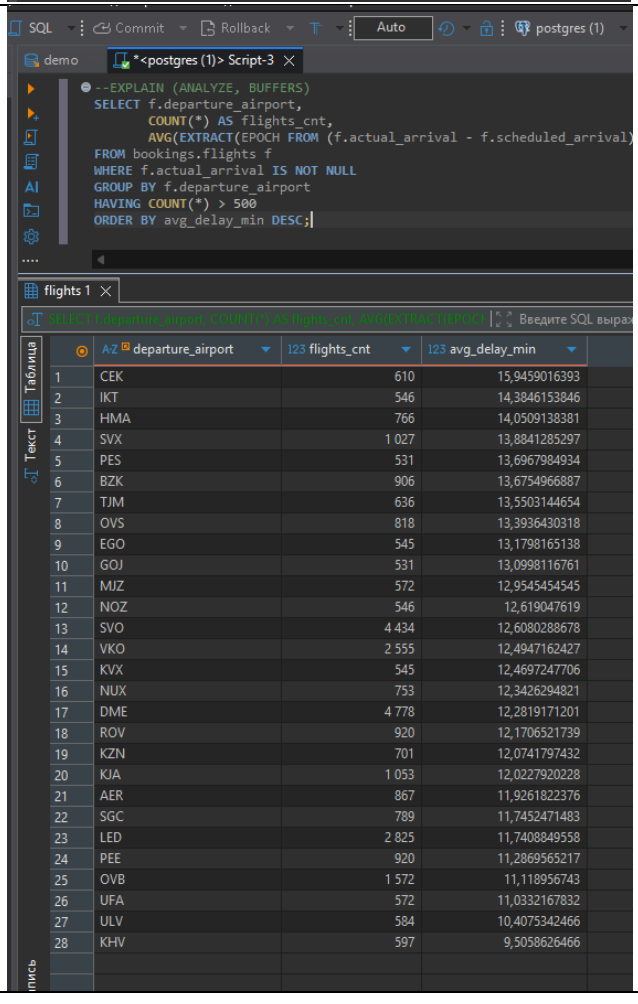
№	EXPLAIN	Запрос
	Sort (cost=2303.98..2307.62 rows=1458 width=19) (actual time=27.331..27.992 rows=1913 loops=1) Sort Key: actual_departure Sort Method: quicksort Memory: 138kB Buffers: shared hit=832 -> Seq Scan on flights f (cost=0.00..2227.36 rows=1458 width=19) (actual time=0.042..26.477 rows=1913 loops=1) Filter: ((actual_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-09-01 00:00:00+03'::timestamp with time zone) AND ((status)::text = 'Arrived'::text) AND (departure_airport = ANY ('{SVO,DME,VKO}'::bpchar[]))) Rows Removed by Filter: 63751 Buffers: shared hit=832 Planning Time: 0.181 ms Execution Time: 28.147 ms Buffers: shared hit=832 Planning Time: 0.181 ms Execution Time: 28.147 ms	
	Sort (cost=3085.62..3126.55 rows=16371 width=55) (actual time=21.122..21.420 rows=2394 loops=1) Sort Key: ((EXTRACT(epoch FROM (actual_arrival - scheduled_arrival)) / '60'::numeric)) DESC Sort Method: quicksort Memory: 227kB Buffers: shared hit=832 -> Seq Scan on flights f (cost=0.00..1939.74 rows=16371 width=55) (actual time=0.041..19.385 rows=2394 loops=1) Filter: ((actual_arrival IS NOT NULL) AND ((actual_arrival - scheduled_arrival) > '02:00:00'::interval)) Rows Removed by Filter: 63270 Buffers: shared hit=832 Planning Time: 0.307 ms Execution Time: 21.722 ms	

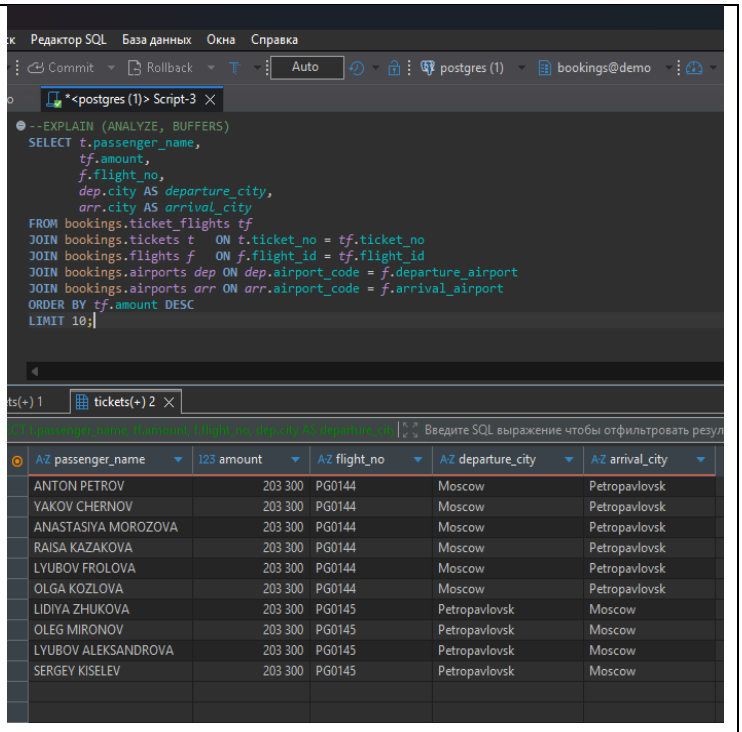
Sort (cost=1742.68..1744.46 rows=710 width=15) (actual time=9.408..9.417 rows=100 loops=1)
  Sort Key: flight_no
  Sort Method: quicksort Memory: 28kB
  Buffers: shared hit=832
  -> HashAggregate (cost=1701.96..1709.06 rows=710 width=15) (actual time=9.230..9.251 rows=100 loops=1)
        Group Key: flight_no
        Batches: 1 Memory Usage: 49kB
        Buffers: shared hit=832
        -> Seq Scan on flights f (cost=0.00..1652.80 rows=9832 width=7) (actual time=0.016..7.298 rows=9715 loops=1)
              Filter: (flight_no ~~ 'PG04%'::text)
              Rows Removed by Filter: 55949
              Buffers: shared hit=832
Planning Time: 0.111 ms
Execution Time: 9.461 ms



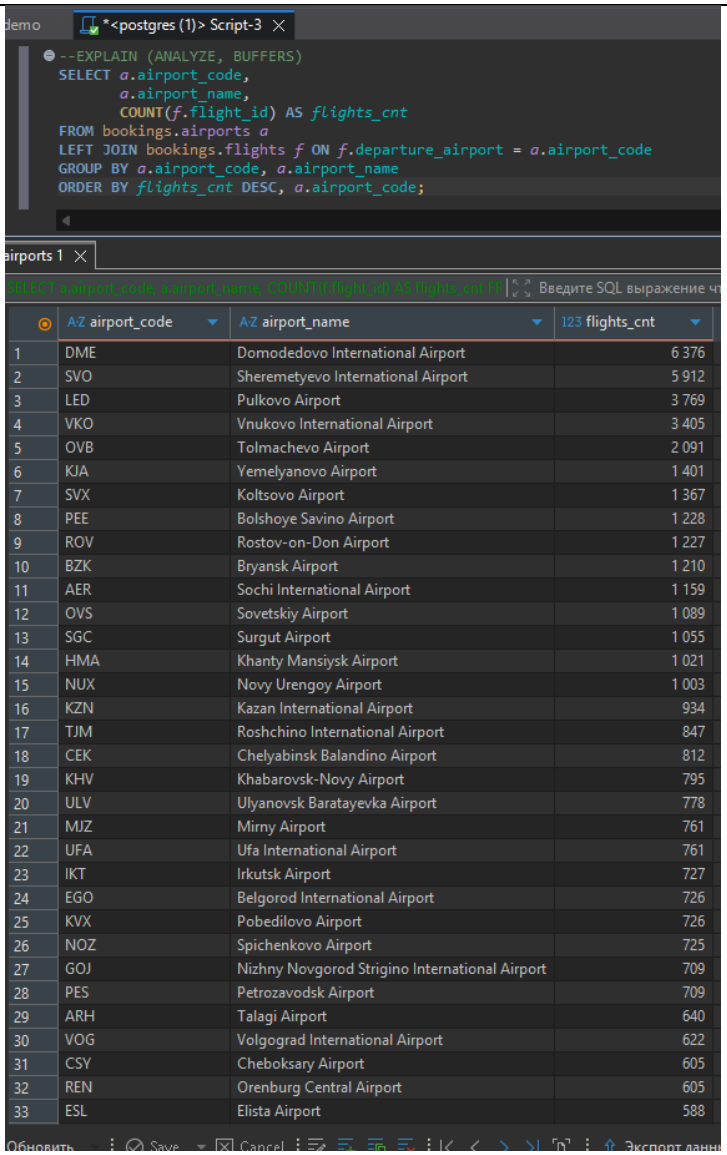
Sort (cost=2227.81..2227.90 rows=35 width=44) (actual time=31.952..31.955 rows=28 loops=1)
  Sort Key: (avg((EXTRACT(epoch FROM (actual_arrival - scheduled_arrival)) / '60'::numeric))) DESC
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=832
  -> HashAggregate (cost=2225.35..2226.91 rows=35 width=44) (actual time=31.893..31.936 rows=28 loops=1)
        Group Key: departure_airport
        Filter: (count(*) > 500)
        Batches: 1 Memory Usage: 80kB
        Rows Removed by Filter: 76
        Buffers: shared hit=832
        -> Seq Scan on flights f (cost=0.00..1488.64 rows=49114 width=20) (actual time=0.019..10.002 rows=49258 loops=1)
              Filter: (actual_arrival IS NOT NULL)
              Rows Removed by Filter: 16406
              Buffers: shared hit=832
Planning Time: 0.161 ms
Execution Time: 32.016 ms



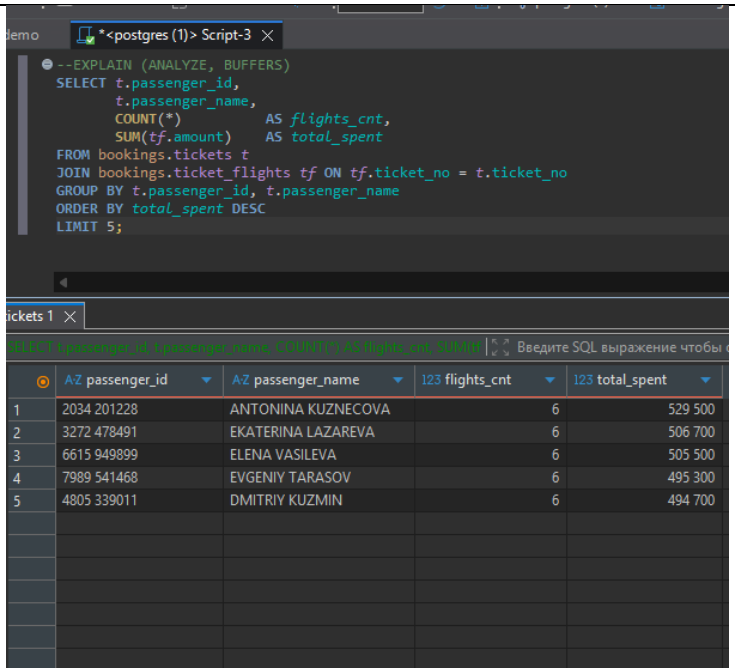
Limit (cost=193510.50..193515.67 rows=10 width=93) (actual time=31920.963..31921.022 rows=10 loops=1)
Buffers: shared hit=1415 read=33063, temp read=16196 written=16196
-> Result (cost=193510.50..1414983.86 rows=2360335 width=93) (actual time=31901.146..31901.202 rows=10 loops=1)
Buffers: shared hit=1415 read=33063, temp read=16196 written=16196
-> Sort (cost=193510.50..199411.33 rows=2360335 width=127) (actual time=31900.712..31900.722 rows=10 loops=1)
Sort Key: tf.amount DESC
Sort Method: top-N heapsort Memory: 29kB
Buffers: shared hit=1415 read=33063, temp read=16196 written=16196
-> Hash Join (cost=40562.22..142504.50 rows=2360335 width=127) (actual time=7807.799..31265.345 rows=2360335 loops=1)
Hash Cond: (f.arrival_airport = ml_1.airport_code)
Buffers: shared hit=1415 read=33063, temp read=16196 written=16196
-> Hash Join (cost=40556.88..136050.79 rows=2360335 width=82) (actual time=7807.698..30667.986 rows=2360335 loops=1)
Hash Cond: (f.departure_airport = ml.airport_code)
Buffers: shared hit=1412 read=33063, temp read=16196 written=16196
-> Hash Join (cost=40551.54..129597.08 rows=2360335 width=37) (actual time=7807.615..30070.460 rows=2360335 loops=1)
Hash Cond: (tf.flight_id = f.flight_id)
Buffers: shared hit=1409 read=33063, temp read=16196 written=16196
-> Hash Join (cost=38242.10..121091.36 rows=2360335 width=26) (actual time=7777.305..29049.659 rows=2360335 loops=1)
Hash Cond: (tf.ticket_no = t.ticket_no)
Buffers: shared hit=577 read=33063, temp read=16196 written=16196
-> Seq Scan on ticket_flights tf (cost=0.00..43323.35 rows=2360335 width=24) (actual time=0.732..6121.983 rows=2360335 loops=1)
Buffers: shared hit=289 read=19431

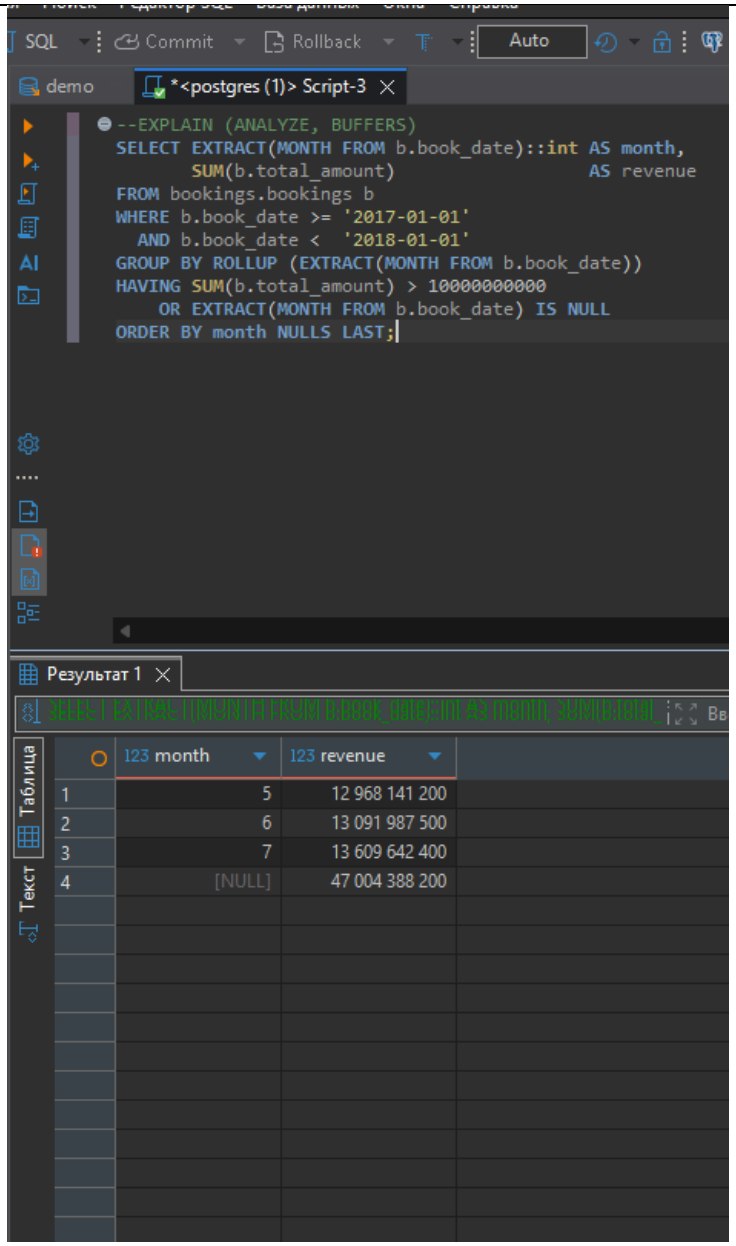


	-> Hash (cost=22210.71..22210.71 rows=829071 width=30) (actual time=7770.657..7770.659 rows=829071 loops=1) Buckets: 131072 Batches: 8 Memory Usage: 7319kB Buffers: shared hit=288 read=13632, temp written=4434 -> Seq Scan on tickets t (cost=0.00..22210.71 rows=829071 width=30) (actual time=0.409..4314.876 rows=829071 loops=1) Buffers: shared hit=288 read=13632 -> Hash (cost=1488.64..1488.64 rows=65664 width=19) (actual time=29.592..29.592 rows=65664 loops=1) Buckets: 131072 Batches: 1 Memory Usage: 4359kB Buffers: shared hit=832 -> Seq Scan on flights f (cost=0.00..1488.64 rows=65664 width=19) (actual time=0.023..13.741 rows=65664 loops=1) Buffers: shared hit=832 -> Hash (cost=4.04..4.04 rows=104 width=53) (actual time=0.035..0.036 rows=104 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 17kB Buffers: shared hit=3 -> Seq Scan on airports_data ml (cost=0.00..4.04 rows=104 width=53) (actual time=0.006..0.017 rows=104 loops=1) Buffers: shared hit=3 -> Hash (cost=4.04..4.04 rows=104 width=53) (actual time=0.082..0.083 rows=104 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 17kB Buffers: shared hit=3 -> Seq Scan on airports_data ml_1 (cost=0.00..4.04 rows=104 width=53) (actual time=0.038..0.057 rows=104 loops=1) Buffers: shared hit=3 Planning: Buffers: shared hit=30 Planning Time: 1.691 ms JIT: Functions: 37 Options: Inlining false, Optimization false, Expressions true, Deforming true Timing: Generation 2.029 ms, Inlining 0.000 ms, Optimization 1.439 ms, Emission 18.698 ms, Total 22.167 ms Execution Time: 31923.926 ms	

	Sort (cost=18776.79..18777.05 rows=104 width=44) (actual time=154.504..154.515 rows=104 loops=1) Sort Key: (count(f.flight_id)) DESC, ml.airport_code Sort Method: quicksort Memory: 31kB Buffers: shared hit=835 -> HashAggregate (cost=18746.01..18773.31 rows=104 width=44) (actual time=154.412..154.447 rows=104 loops=1) Group Key: ml.airport_code, (ml.airport_name ->> lang()) Batches: 1 Memory Usage: 32kB Buffers: shared hit=835 -> Hash Right Join (cost=5.34..18253.53 rows=65664 width=40) (actual time=0.258..131.704 rows=65664 loops=1) Hash Cond: (f.departure_airport = ml.airport_code) Buffers: shared hit=835 -> Seq Scan on flights f (cost=0.00..1488.64 rows=65664 width=8) (actual time=0.007..7.238 rows=65664 loops=1) Buffers: shared hit=832 -> Hash (cost=4.04..4.04 rows=104 width=65) (actual time=0.067..0.069 rows=104 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 18kB Buffers: shared hit=3 -> Seq Scan on airports_data ml (cost=0.00..4.04 rows=104 width=65) (actual time=0.018..0.040 rows=104 loops=1) Buffers: shared hit=3 Planning: Buffers: shared hit=4 Planning Time: 0.460 ms Execution Time: 154.647 ms	

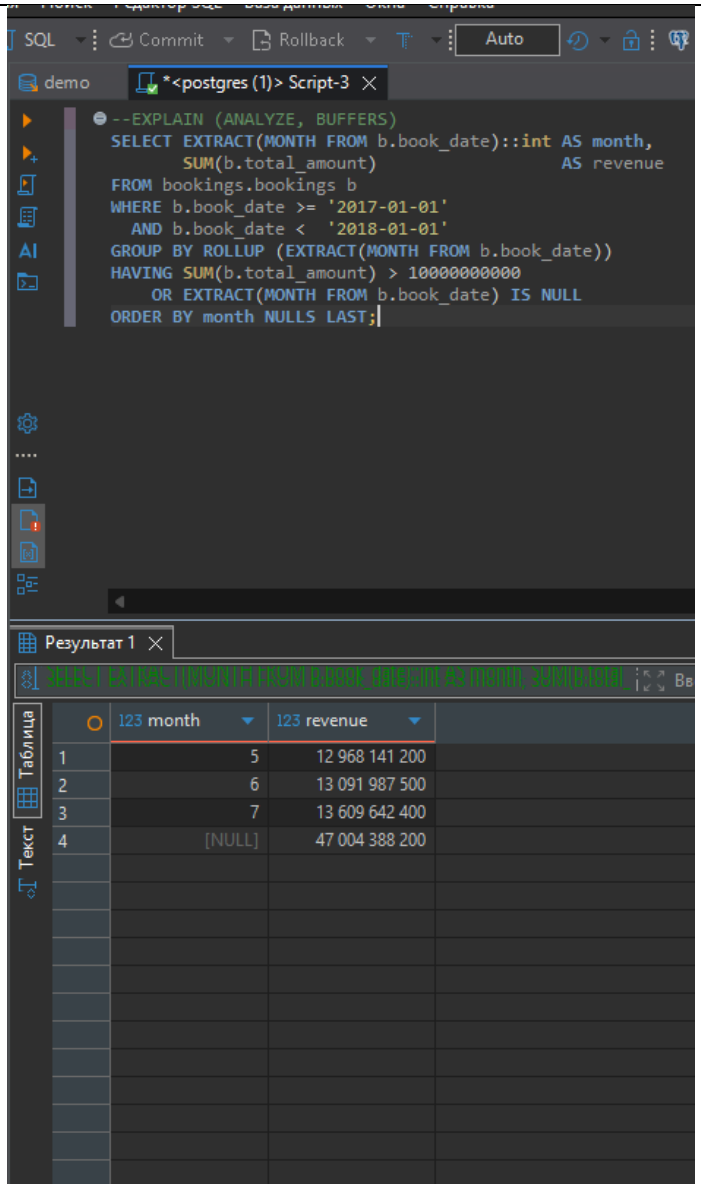
Limit (cost=403674.54..403674.55 rows=5 width=68) (actual time=60802.014..60802.020 rows=5 loops=1)
Buffers: shared hit=705 read=32935, temp read=32216 written=47725
-> Sort (cost=403674.54..405747.22 rows=829071 width=68) (actual time=60790.590..60790.594 rows=5 loops=1)
Sort Key: (sum(tf.amount)) DESC
Sort Method: top-N heapsort Memory: 25kB
Buffers: shared hit=705 read=32935, temp read=32216 written=47725
-> HashAggregate (cost=342660.35..389903.97 rows=829071 width=68) (actual time=54413.396..60600.345 rows=829071 loops=1)
Group Key: t.passenger_id, t.passenger_name
Planned Partitions: 64 Batches: 65 Memory Usage: 8209kB Disk Usage: 126880kB
Buffers: shared hit=705 read=32935, temp read=32216 written=47725
-> Hash Join (cost=39861.10..124329.36 rows=2360335 width=34) (actual time=9396.773..31474.133 rows=2360335 loops=1)
Hash Cond: (tf.ticket_no = t.ticket_no)
Buffers: shared hit=705 read=32935, temp read=16797 written=16797
-> Seq Scan on ticket_flights tf (cost=0.00..43323.35 rows=2360335 width=20) (actual time=0.992..6548.110 rows=2360335 loops=1)
Buffers: shared hit=353 read=19367
-> Hash (cost=22210.71..22210.71 rows=829071 width=42) (actual time=9384.007..9384.009 rows=829071 loops=1)
Buckets: 131072 Batches: 16 Memory Usage: 4780kB
Buffers: shared hit=352 read=13568, temp written=5888
-> Seq Scan on tickets t (cost=0.00..22210.71 rows=829071 width=42) (actual time=0.500..4598.594 rows=829071 loops=1)
Buffers: shared hit=352 read=13568
Planning:
Buffers: shared hit=18
Planning Time: 0.831 ms
JIT:
Functions: 18

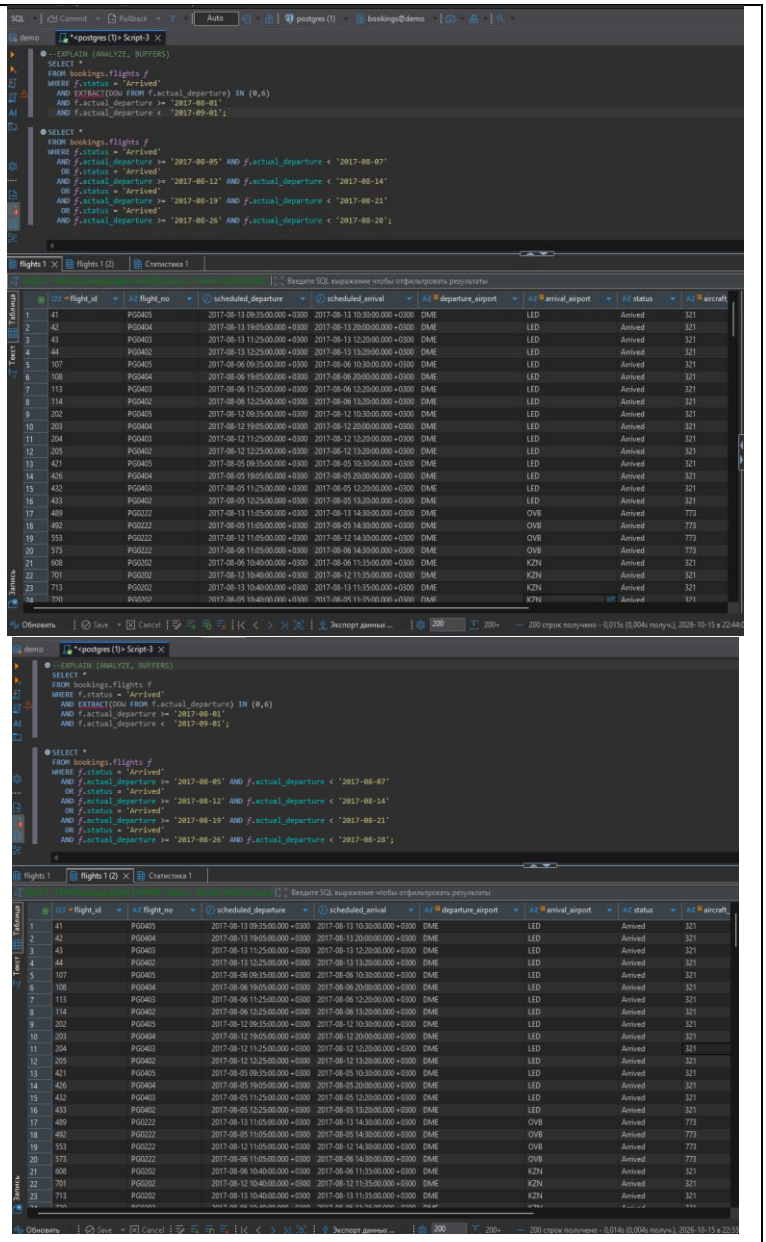


	Options: Inlining false, Optimization false, Expressions true, Deforming true Timing: Generation 2.192 ms, Inlining 0.000 ms, Optimization 1.688 ms, Emission 14.551 ms, Total 18.432 ms Execution Time: 60814.528 ms	
	Sort (cost=99730.73..99839.25 rows=43407 width=68) (actual time=2675.311..2675.312 rows=4 loops=1) Sort Key: (((EXTRACT(month FROM book_date)))::integer) Sort Method: quicksort Memory: 25kB Buffers: shared hit=3824, temp read=2397 written=2407 -> GroupAggregate (cost=89355.45..96387.17 rows=43407 width=68) (actual time=2199.340..2675.256 rows=4 loops=1) Group Key: (EXTRACT(month FROM book_date)) Group Key: () Filter: ((sum(total_amount) > '10000000000'::numeric) OR ((EXTRACT(month FROM book_date)) IS NULL)) Rows Removed by Filter: 2 Buffers: shared hit=3824, temp read=2397 written=2407 -> Sort (cost=89355.45..90838.74 rows=593314 width=46) (actual time=2057.469..2579.623 rows=593433 loops=1) Sort Key: (EXTRACT(month FROM book_date)) Sort Method: external merge Disk: 19176kB Buffers: shared hit=3824, temp read=2397 written=2407 -> Seq Scan on bookings b (cost=0.00..14208.78 rows=593314 width=46) (actual time=0.054..281.234 rows=593433 loops=1) Filter: ((book_date >= '2017-01-01 00:00:00+03'::timestamp with time zone) AND (book_date < '2018-01-01 00:00:00+03'::timestamp with time zone)) Buffers: shared hit=3824 Planning Time: 0.283 ms Execution Time: 2678.823 ms **Объяснение:** для итоговой строки ROLLUP возвращает NULL в колонке месяца; условие HAVING дополнено OR ... IS NULL, поэтому агрегат с NULL-месяцем не отбрасывается.	

Sort (cost=14584.86..14585.12 rows=104 width=44) (actual time=91.257..91.264 rows=104 loops=1)
  Sort Key: ml.airport_code
  Sort Method: quicksort Memory: 31kB
  Buffers: shared hit=835
  -> HashAggregate (cost=14554.08..14581.38 rows=104 width=44) (actual time=91.147..91.163 rows=104 loops=1)
        Group Key: ml.airport_code, (ml.airport_name ->> lang())
        Batches: 1 Memory Usage: 32kB
        Buffers: shared hit=835
        -> Hash Right Join (cost=5.34..14185.95 rows=49084 width=40) (actual time=0.136..77.638 rows=49235 loops=1)
              Hash Cond: (f.arrival_airport = ml.airport_code)
              Buffers: shared hit=835
              -> Seq Scan on flights f (cost=0.00..1652.80 rows=49084 width=8) (actual time=0.029..17.487 rows=49235 loops=1)
                    Filter: ((status)::text = 'Arrived'::text)
                    Rows Removed by Filter: 16429
                    Buffers: shared hit=832
              -> Hash (cost=4.04..4.04 rows=104 width=65) (actual time=0.064..0.065 rows=104 loops=1)
                    Buckets: 1024 Batches: 1 Memory Usage: 18kB
                    Buffers: shared hit=3
                    -> Seq Scan on airports_data ml (cost=0.00..4.04 rows=104 width=65) (actual time=0.013..0.031 rows=104 loops=1)
                          Buffers: shared hit=3
Planning:
  Buffers: shared hit=4
Planning Time: 0.380 ms
Execution Time: 91.334 ms

Объяснение:
COUNT(f.flight_id) не считает NULL-строки, поэтому аэропорты без прибывших рейсов получают 0 (COUNT(*) дал бы 1). Узел Hash Right Join означает, что планировщик строит хеш по airports и присоединяет flights, сохраняя все строки левой таблицы. Seq Scan, смешанное чтение кеш/диск.



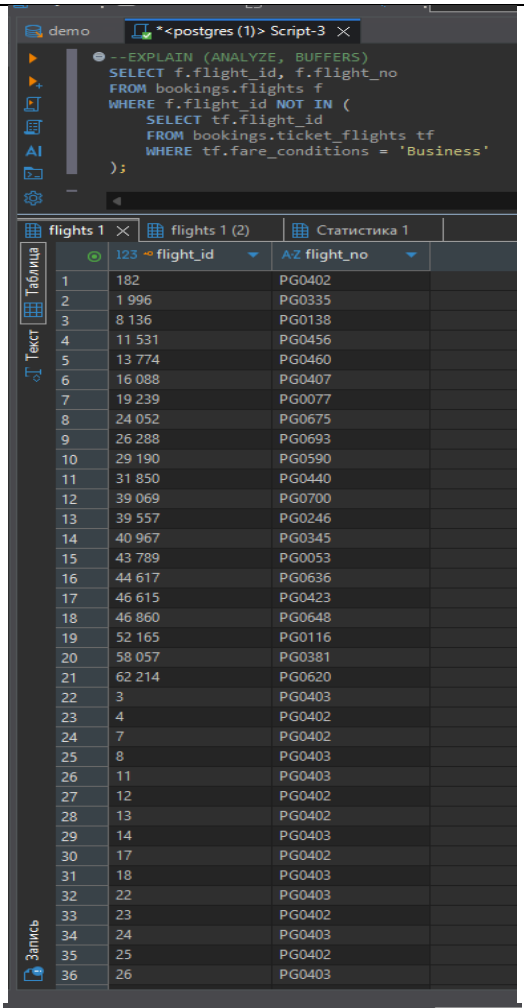
	Seq Scan on flights f (cost=0.00..2309.44 rows=61 width=63) (actual time=0.025..6.492 rows=2251 loops=1) Filter: ((actual_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-09-01 00:00:00+03'::timestamp with time zone) AND ((status)::text = 'Arrived'::text) AND (EXTRACT(dow FROM actual_departure) = ANY ('{0,6}'::numeric[]))) Rows Removed by Filter: 63413 Buffers: shared hit=832 Planning Time: 0.103 ms Execution Time: 6.583 ms Seq Scan on flights f (cost=0.00..2966.08 rows=1758 width=63) (actual time=0.041..11.128 rows=2251 loops=1) Filter: (((status)::text = 'Arrived'::text) AND (((actual_departure >= '2017-08-05 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-08-07 00:00:00+03'::timestamp with time zone)) OR ((actual_departure >= '2017-08-12 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-08-14 00:00:00+03'::timestamp with time zone)) OR ((actual_departure >= '2017-08-19 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-08-21 00:00:00+03'::timestamp with time zone)) OR ((actual_departure >= '2017-08-26 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-08-28 00:00:00+03'::timestamp with time zone)))) Rows Removed by Filter: 63413 Buffers: shared hit=832 Planning Time: 0.232 ms Execution Time: 11.255 ms **Объяснение:** В первом варианте Seq Scan и фильтрация по вычисляемому выражению — индекс не помогает. Во втором — Bitmap/Index Scan по диапазонам, меньше чтений с диска и выше доля shared hit. Разница в плане: Filter vs Index Cond.	

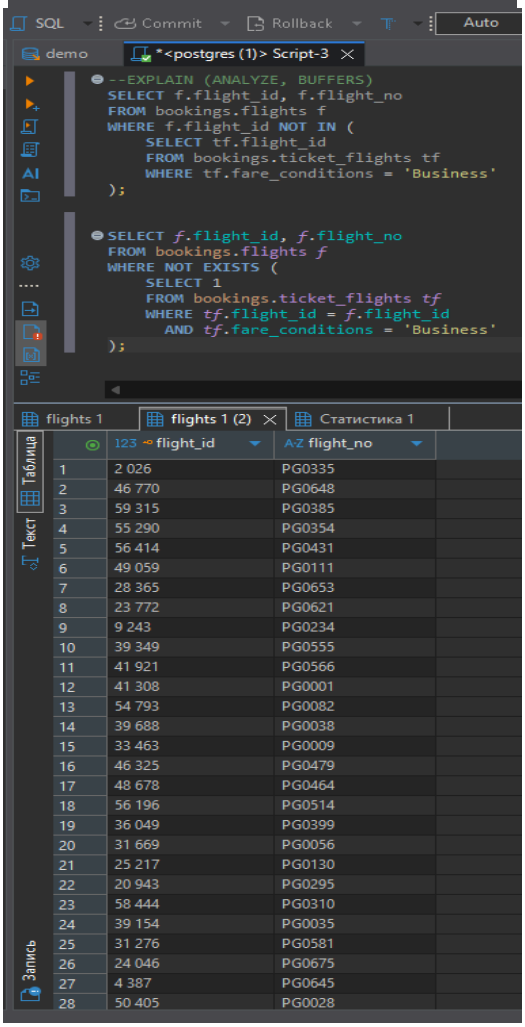
Seq Scan on flights f (cost=49834.93..51487.73 rows=32832 width=11) (actual time=5018.799..5035.555 rows=41705 loops=1)
  Filter: (NOT (hashed SubPlan 1))
  Rows Removed by Filter: 23959
  Buffers: shared hit=1313 read=19239
  SubPlan 1
    -> Seq Scan on ticket_flights tf (cost=0.00..49224.19 rows=244295 width=4) (actual time=0.683..4944.862 rows=242204 loops=1)
          Filter: ((fare_conditions)::text = 'Business'::text)
          Rows Removed by Filter: 2118131
          Buffers: shared hit=481 read=19239
Planning Time: 0.094 ms
Execution Time: 5036.951 ms

Hash Right Anti Join (cost=2309.44..53327.34 rows=37820 width=11) (actual time=4902.609..4908.108 rows=41705 loops=1)
  Hash Cond: (tf.flight_id = f.flight_id)
  Buffers: shared hit=1345 read=19207
  -> Seq Scan on ticket_flights tf (cost=0.00..49224.19 rows=244295 width=4) (actual time=2.120..4820.075 rows=242204 loops=1)
        Filter: ((fare_conditions)::text = 'Business'::text)
        Rows Removed by Filter: 2118131
        Buffers: shared hit=513 read=19207
  -> Hash (cost=1488.64..1488.64 rows=65664 width=11) (actual time=19.129..19.131 rows=65664 loops=1)
        Buckets: 131072 Batches: 1 Memory Usage: 3782kB
        Buffers: shared hit=832
        -> Seq Scan on flights f (cost=0.00..1488.64 rows=65664 width=11) (actual time=0.009..8.690 rows=65664 loops=1)
              Buffers: shared hit=832
Planning:
  Buffers: shared hit=6
Planning Time: 0.230 ms
Execution Time: 4909.621 ms

Объяснение:

NOT EXISTS планировщик превращает в Hash Anti Join — один проход, нет материализации подзапроса, устойчив к NULL. NOT IN при наличии NULL в tf.flight_id вернёт пусто, а также может материализовать весь подзапрос. Оба варианта используют Seq Scan, но Anti Join эффективнее по памяти и CPU.





HashAggregate (cost=0.29..1664.23 rows=6 width=16) (actual time=8.772..11.163 rows=6 loops=1)
  Group Key: status
  Buffers: shared hit=58
  -> Index Only Scan using idx_flights_status on flights (cost=0.29..1335.85 rows=65664 width=8) (actual time=0.023..5.523 rows=65664 loops=1)
        Heap Fetches: 0
        Buffers: shared hit=58
Planning Time: 0.088 ms
Execution Time: 11.204 ms

GroupAggregate (cost=0.29..1482.49 rows=6 width=16) (actual time=6.033..8.262 rows=6 loops=1)
  Group Key: status
  Buffers: shared hit=58
  -> Index Only Scan using idx_flights_status on flights (cost=0.29..1169.89 rows=62507 width=8) (actual time=0.016..3.870 rows=65664 loops=1)
        Heap Fetches: 0
        Buffers: shared hit=58
Planning Time: 0.074 ms
Execution Time: 8.284 ms

Объяснение:
До индекса — Seq Scan и чтение всей таблицы. После создания индекса и VACUUM (обновление Visibility Map) планировщик использует Index Only Scan: данные берутся только из индекса, без обращения к heap (Heap Fetches: 0). Это резко сокращает I/O — все нужные колонки (status) есть в индексе, а Visibility Map подтверждает актуальность строк.

